```
import pandas as pd
df=pd.read_csv("/content/creditcard.csv")
```

```
df.columns
```

```
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
       'Class'],
      dtype='object')
```

```
df.head(4)
```

	Time	V1	V2	V3	V4	V5	V6	V7
0	0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609

4 rows × 31 columns

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Drop rows with NaN in 'Class' column before splitting X and y
df_cleaned = df.dropna(subset=['Class'])

X = df_cleaned.drop("Class", axis=1)
y = df_cleaned["Class"]

scaler = StandardScaler()
X["Amount"] = scaler.fit_transform(X[["Amount"]])

X = X.drop("Time", axis=1)
```

```
y.value_counts()
```

Class	count
0.0	45503
1.0	142

dtype: int64

```
y = y.dropna()
```

```
X_train, X_test, y_train,
    X, y, test_size=0.2,
)
```

```
from sklearn.utils import class_weight
import numpy as np

weights = class_weight.compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)

class_weights = {0: weights[0], 1: weights[1]}
print(class_weights)
```

```
{0: np.float64(0.501565848030328), 1: np.float64(160.1578947368421)}
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam, SGD, RMSprop
from sklearn.metrics import f1_score
```

```
batch_sizes = [32, 64]
hidden_layers_list = [3, 5, 7]
optimizers = {
    "adam":lambda:Adam(),
    "sgd":lambda: SGD()

}

results = []
```

```
for batch in batch_sizes:
    for layers in hidden_layers_list:
        for opt_name, opt_fn in optimizers.items():

            print(f"\nTraining: batch={batch}, layers={layers}, optimizer={o


            model = Sequential()
            model.add(Dense(64, input_dim=X_train.shape[1], activation='relu

            for _ in range(layers):
                model.add(Dense(32, activation='relu'))
                model.add(Dropout(0.3))

            model.add(Dense(1, activation='sigmoid'))
```

```
            optimizer = opt_fn()
            model.compile(
                loss='binary_crossentropy',
                optimizer=optimizer,
                metrics=['accuracy']
            )


            model.fit(
                X_train, y_train,
                epochs=10,
                batch_size=batch,
                validation_split=0.2,
                class_weight=class_weights,
                verbose=0
            )

            # Predict
            y_pred = (model.predict(X_test) > 0.5).astype(int)

            # Use F1 score (IMPORTANT)
            f1 = f1_score(y_test, y_pred)

            results.append({
                "batch_size": batch,
                "layers": layers,
                "optimizer": opt_name,
                "f1_score": f1
            })

            print("F1 Score:", f1)
```

```
Training: batch=32, layers=3, optimizer=adam
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
F1 Score: 0.6582278481012658

Training: batch=32, layers=3, optimizer=sgd
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
F1 Score: 0.6506024096385542

Training: batch=32, layers=5, optimizer=adam
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
F1 Score: 0.5869565217391305

Training: batch=32, layers=5, optimizer=sgd
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
F1 Score: 0.6666666666666666
```

```
Training: batch=32, layers=7, optimizer=adam
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
F1 Score: 0.45045045045045046

Training: batch=32, layers=7, optimizer=sgd
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
F1 Score: 0.0

Training: batch=64, layers=3, optimizer=adam
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
F1 Score: 0.7105263157894737

Training: batch=64, layers=3, optimizer=sgd
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
F1 Score: 0.32335329341317365

Training: batch=64, layers=5, optimizer=adam
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
286/286 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
F1 Score: 0.6046511627906976

Training: batch=64, layers=5, optimizer=sgd
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
```

```
results_df = pd.DataFrame(results)
print("\nBest Configurations:")
print(results_df.sort_values(by="f1_score", ascending=False).h
```

```
Best Configurations:
   batch_size  layers optimizer  f1_score
6          64       3      adam  0.710526
3          32       5       sgd  0.666667
0          32       3      adam  0.658228
1          32       3       sgd  0.650602
8          64       5      adam  0.604651
```